

Forward Production Scheduler

JK Tyre — Banmore Tyre Plant

1. Background

Banmore Tyre Plant manufactures Passenger-Car Radial tyres. A finished tyre is the output of a long chain of operations — Mixing, Final Mixing, Calendering, Cutting, Ply Cutting, Bead Building, Extrusion, Tyre Building, and finally Curing — running across many machines with their own setup, processing and aging characteristics.

Curing is the last operation. The plant publishes a curing schedule — a fixed assignment of SKUs to presses across time — that drives every upstream demand. Today, planners build the upstream schedule manually: deciding how many lots of each compound to mix, when to start each calendering run, which machine to send each ply to, and so on. This is slow, error-prone, and frequently leaves either green tyres aging past their window or curing presses idle waiting for green tyres.

The plant needs an automated, deterministic, BOM-driven engine that takes the curing plan as a fixed input and produces a fully time-stamped, machine-assigned schedule for every upstream operation.

2. Problem statement

The problem

Given a fixed curing schedule, a Bill of Materials, a routing, and operational masters for aging, MPQ, and item types — produce a complete forward production schedule for the pilot SKU that respects every hard constraint, places every lot on an eligible machine, and is reproducible and explainable.

2.1 What you are NOT being asked to do

- You are NOT building an optimiser that re-arranges the curing schedule. Curing is a hard input.
- You are NOT being asked to predict demand. Demand is whatever the curing plan dictates.
- You are NOT being asked to model labour, energy, or cost — only material flow, machine assignment and time.

2.2 The pilot scope

- One Passenger-Car Radial SKU (the pilot SKU) — given in the curing CSV.
- The supplied May curing horizon.
- All components per tyre must be scheduled, EXCEPT the Capstrip chain (its BOM is incomplete in the supplied data and is out of scope for this phase).

3. Inputs you will receive

You will be given the following raw files. You may NOT modify them — your engine must consume them as-is and document any data-quality gaps it encounters.

3.1 Curing schedule (CSV)

A row per curing block over the planning horizon, with columns indicating date, shift, press, SKU code, start and end time, quantity, cycle time, and any pre-existing green-tyre inventory. Treat each row as a non-negotiable demand point.

3.2 BOM (Bill of Materials)

A multi-level parent-child relationship table linking every finished item back to its components and raw materials. The pilot SKU's BOM is rooted at the green tyre item and spans seven levels of depth. Each row gives the consumption quantity in a specific UOM.

3.3 Routing

For every produced item the routing table provides: operation name, department, eligible machines (one or many), process time and its UOM (SEC/BATCH, MIN, M/MIN, SEC, MIN with batch size, etc.), and optionally a transfer time. The routing is your source of truth for what machines an operation can run on and how long it takes.

3.4 Operational masters

- Aging master — per ItemCode, the minimum and maximum aging interval allowed between production of that item and its consumption downstream. Units vary (Days, Hours, Minutes) and must be normalised.
- MPQ master — per item type, the minimum and maximum production quantity allowed per lot. Lots below MPQ_Min are not viable; lots above MPQ_Max should be split.
- Item-type master — maps each ItemCode to its category (Master Compound, Final Compound, Inner Liner, Rubberized Ply, etc.). Used to look up MPQ and to drive lot-sizing rules.

4. Hard constraints the schedule MUST satisfy

Your engine's output is correct only if every one of these holds for every lot in the schedule:

4.1 Aging window

- No consumer lot may start before its predecessor's MINIMUM aging window has elapsed (the material is not yet usable).
- No consumer lot may start after its predecessor's MAXIMUM aging window has closed (the material has expired).
- Both ends apply to every BOM edge, including the final Building → Curing handoff.

4.2 BOM completeness (AND-join)

- An operation that consumes N components cannot start until ALL N components have at least one ready-and-aged lot available.
- The Tyre Building operation consumes eight distinct components — every one of them is mandatory.

4.3 Machine eligibility

- Every lot must run on a machine listed in the routing for that operation. The engine may not place a lot on any other machine.
- A machine can only run one lot at a time.

4.4 MPQ

- Every lot must satisfy MPQ_Min for its item type.
- Lots should respect MPQ_Max; splitting one large demand into multiple lots is acceptable.

4.5 Curing is fixed

- The curing schedule is INPUT, not output. Your engine may not move a curing block.
- If aging or AND-join constraints cannot be satisfied without violating the curing plan, you must FLAG the violation explicitly and continue — do NOT silently shift curing.

4.6 Determinism

- Running the engine twice with the same inputs must produce the same schedule. There must be no randomness or non-reproducible behaviour.

5. Plant policies (soft rules)

- Treat plants as running 24x7 in this phase — do not model shift breaks or planned-maintenance windows.
- Apply a flat 15-minute changeover between different products on the same machine, and 0 minutes for the same product back-to-back.
- Apply a flat 10-minute transfer time between a producer and its consumer.
- Use a single 95% efficiency factor across all machines.
- Tyre Building has multiple parallel machines available, but production-floor preference is to commit to ONE Building machine per product family and only spill to a second/third when staying on the primary would force a component to age past its window.
- Process times with null cells in the data must be filled with a documented, justifiable default.

6. Deliverables

6.1 Code

- A working, documented codebase (any language; Python recommended for data tooling).
- Modular structure — each logical step (demand explosion, lot sizing, graph construction, time calculation, scheduling, KPI analysis) should be runnable in isolation.
- All inputs read from the supplied raw files; all outputs written to a single dated outputs folder.

- A README that explains how to run the pipeline end-to-end.

6.2 Schedule outputs

- A lot-level schedule with one row per scheduled production lot: lot ID, item code, item type, machine, start datetime, end datetime, duration, quantity, UOM, the demand date it serves, and an on-time flag.
- A machine-level view (the same lots sorted by machine and time) for floor-level execution.
- A dedicated Building → Curing handoff report classifying every Building lot as OK / LATE / EARLY (or equivalent) so the planner can adjust either side of the handoff.

6.3 Diagnostics

- An aging-violations sheet listing every consumer-predecessor pair where the aging window was breached.
- An infeasibilities sheet listing any lot the engine could not schedule and the reason.
- A KPI report: total lots scheduled, OTIF %, total changeover and processing minutes, schedule span, machine utilisation, and any other metric you think is important.

6.4 Visualisation

- A static visualisation (HTML / PNG / SVG) of the operation graph for the pilot SKU showing departments, precedence and aging windows.
- A Gantt-style view of the schedule for at least three sample curing blocks — early, middle and late in the horizon.

6.5 Document

- A short methodology write-up (4–8 pages) describing your approach, the assumptions you took, and the limitations of your current build. Include a brief discussion of any design choices that were close calls.

7. Acceptance criteria

Your submission will be judged on these criteria in this order:

1. Correctness — every hard constraint in Section 4 must hold for every scheduled lot. Violations must be detected and reported by your engine itself, not just by a reviewer.
2. Reproducibility — re-running the engine on the same inputs must produce identical outputs.
3. Explainability — for any scheduled lot, you must be able to point to the data that justified its machine choice, start time, and quantity.
4. Coverage — the schedule must include every operation in the pilot SKU's BOM (excluding the Capstrip chain), down to the master compound.
5. Diagnostics quality — the Building→Curing handoff report, aging-violation report and infeasibility report must be complete and actionable.
6. Code quality — clean structure, readable, runnable from a fresh clone with documented steps.
7. Visualisation clarity — the graph and Gantt views should be readable without a separate legend.

8. Suggested approach (your call, but use this as a starting checklist)

- Before writing any scheduling code, AUDIT the raw inputs and document every gap — null process times, ambiguous BOM rows, encoding glitches, UOM inconsistencies.
- Decide explicitly how you will handle each gap. Defaults must be justified, not invented.
- Separate the engine into stages with clear, inspectable intermediate outputs. You will iterate; you cannot iterate if you cannot see what each stage produced.
- Build the smallest pipeline that produces a runnable schedule first, then layer in constraints and diagnostics.
- Treat the aging window as a HARD constraint from day one. Adding it late forces re-design.
- Visualise early and often — graphs and Gantt views surface bugs your console will not.

9. Open questions worth thinking about

- How will you dispatch when many lots are ready at once? Earliest demand date? Longest downstream path? Something else? Justify your choice.
- How will you size lots — by demand window, by MPQ band, by aging window, or some combination?
- If a component has many committed lots, which one do you pick to feed a given consumer? Pick the rule and explain why.
- What is your engine's behaviour when the constraint set is infeasible — does it abort, partial-schedule, or relax constraints in a documented order?
- Is your output stable when inputs change slightly? A one-row change to the curing plan should not shuffle the entire upstream schedule.

10. Ground rules

- Work independently. You may consult your peers, but the submission must be your own.
- Do NOT look at any prior or current solution within the company. Approach the problem fresh.
- Ask the planning team for clarifications on the data — they ARE the source of truth. Do NOT ask another engineer for design hints.
- Document EVERY assumption. A wrong-but-documented assumption is recoverable; an undocumented assumption is a defect.
- Commit to a public git repository so progress is visible from Day 1.

11. What good looks like

- Your engine runs end-to-end on a fresh clone with a single command.
- Every scheduled lot points back to the raw-data row that defined its time, quantity and machine choice.
- Aging violations and infeasibilities are detected, counted and reported — not hidden.

- Re-running with a small change to the curing plan produces a small, explainable change to the schedule.
- The planning team can pick up your KPI report and immediately see which component is the actual bottleneck — and which lever to pull next.

Good luck. The hardest part is reading the data twice before writing any code.